

API Planning for To-Do List Application

CRUD Operations Overview These APIs collectively fulfill the complete CRUD lifecycle for a To-Do List application.

- **Create:** Achieved via the POST method to add new tasks.
- **Read:** Achieved via the GET method to fetch existing tasks.
- **Update:** Achieved via the PUT method to modify task details or completion status.
- **Delete:** Achieved via the DELETE method to remove tasks from the database.

1. Add Task (Create)

- **API Endpoint:** /api/tasks
- **HTTP Method:** POST
- **Request Body Format:**

JSON

```
{  
  "title": "Complete React Assignment",  
  "description": "Finish Task 36 and submit the zip file"  
}
```

- **Unique Identifier Logic:** A unique ID (such as a UUID) will be automatically generated by the backend server or database upon creation. The client does not need to provide it.
- **Expected Response:** 201 Created

JSON

```
{  
  "id": "a1b2c3d4",  
  "title": "Complete React Assignment",  
  "description": "Finish Task 36 and submit the zip file",  
  "completed": false  
}
```

- **Reasoning:** Following RESTful design principles, creating a new resource under a collection uses the POST method on the base collection URL.

2. Get All Tasks (Read)

- **API Endpoint:** /api/tasks
- **HTTP Method:** GET
- **Request Body Format:** None required.
- **Expected Response:** 200 OK

JSON

```
[  
  {  
    "id": "a1b2c3d4",  
    "title": "Complete React Assignment",  
    "description": "Finish Task 36 and submit the zip file",  
    "completed": false  
  }  
]
```

- **Reasoning:** A GET request to the base collection URL is the standard RESTful approach to retrieve an array containing all resources.

3. Update a Task (Update)

- **API Endpoint:** /api/tasks/:id
- **HTTP Method:** PUT
- **Request Body Format:**

JSON

```
{  
  "title": "Complete React Assignment",  
  "description": "Finish Task 36 and submit the zip file",  
  "completed": true  
}
```

- **Expected Response:** 200 OK

JSON

```
{  
  "id": "a1b2c3d4",  
  "title": "Complete React Assignment",  
  "description": "Finish Task 36 and submit the zip file",  
  "completed": true  
}
```

- **Reasoning:** We use the `:id` path parameter to specify exactly which task is being modified. PUT is used because we are completely replacing or updating the properties of that specific resource.

4. Delete a Task (Delete)

- **API Endpoint:** `/api/tasks/:id`
- **HTTP Method:** DELETE
- **Request Body Format:** None required.
- **Expected Response:** 200 OK (or 204 No Content)

JSON

```
{  
  "message": "Task successfully deleted"  
}
```

- **Reasoning:** DELETE targeting a specific `:id` URL is the strict REST standard for safely destroying a single resource without affecting the rest of the collection.

Potential Challenges in Implementation

1. **Data Validation:** Ensuring the user does not send empty titles or incorrect data types (e.g., passing a string instead of a boolean for the completed status).
2. **Error Handling:** Properly catching and returning 404 Not Found errors if a user tries to Update or Delete an `:id` that does not currently exist in the backend database.
3. **State Synchronization:** Ensuring the frontend React state updates perfectly in sync with the backend database state after asynchronous API calls resolve.